

SynAT.Vis — User Guide

A transcript-level **red-flag scanner** for recombinant gene cassettes destined for **nuclear** expression in *Chlamydomonas reinhardtii*. Give it a designed sequence; it tells you *why the mRNA might fail* — premature poly(A) signals, silencing-risk GC troughs, rare-codon clusters, cloning sites, upstream AUGs, destabilising elements — and proposes a synonymous fix for each. It is **diagnostic, not predictive**: it never emits an "expression score", it is **silent on protein stability**, and a nuclear report says nothing about a chloroplast construct.

This guide gets it running on any computer (Windows, macOS, Linux).

1. Requirements

- **Python 3.8 or newer.** That is the only hard requirement — the core uses the standard library alone. Check what you have:

OS	Command
Windows	<code>py --version</code>
macOS / Linux	<code>python3 --version</code>

No Python? Install from <https://www.python.org/downloads/> (on Windows tick **"Add python.exe to PATH"**), or `brew install python` (macOS) / your package manager (Linux).

- Optional feature backends (the tool works without them and says so when one is missing): **pyyaml** (faster config parsing), **biopython** (GenBank/SnapGene input), **ViennaRNA** (real 5'UTR folding).

Throughout this guide, use `py` on Windows and `python3` on macOS/Linux wherever you see `python`.

2. Install

Unzip the project somewhere, then open a terminal **in the project folder** (the one containing `pyproject.toml`).

Option A — Zero install (fastest)

The core needs no packages. Just run it in place:

```
python -m synatvis profiles
```

If that lists two profiles, you are done — skip to §3.

Option B — Install the `synatvis` command (recommended)

A virtual environment keeps this isolated from your system Python.

Windows (PowerShell):

```
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install -e .
```

(If activation is blocked: run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, or skip activation and call `.venv\Scripts\synatvis.exe` directly.)

macOS / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

Now `synatvis` works as a command anywhere while the venv is active. Add optional backends with `pip install -e ".[full]"` (or just `".[yaml]"`). `-e` is an *editable* install — edits to the code take effect with no reinstall.

In the examples below, `synatvis ...` (installed) and `python -m synatvis ...` (zero-install, from the project folder) are interchangeable.

3. Verify it works

One-click option (no terminal): double-click `RUN_ME.bat` on Windows, or `RUN_ME.command` on macOS (first time, run `chmod +x RUN_ME.command` once).

It finds Python, runs the self-test, and shows the result — you want

`6/6 checks passed.`

From a terminal, two independent checks:

```
python selftest.py
```

Scans the six test-kit cassettes and confirms each planted problem is caught and the clean control stays quiet → `6/6 checks passed.`

```
python -m synatvis validate leg2
```

Injects known motifs into a clean parent and confirms detection → all `[PASS]`.

(If you installed `pytest`, `python -m pytest -q` runs the full 14-test suite.)

4. Scan your own cassette

Easiest (no typing): drag your `.fasta` file onto `SCAN-a-gene.bat`

(Windows), or double-click `SCAN-a-gene.command` (macOS) and pick your file.

You get a plain-language report on screen, also saved next to your file. The

rest of this section is the terminal equivalent; add `--plain` to any scan

for the plain-language version.

Step 1 — save your design as FASTA (`.fasta` / `.fa`), one sequence:

```
>my_transgene
ATGGCC...TAA
```

GenBank/SnapGene `.gb/.gbk` also works if Biopython is installed.

Step 2 — find the CDS coordinates. If the file is coding sequence only, skip

this. If it includes UTRs, note where the CDS starts and ends (0-based,

half-open), e.g. a 30-nt 5'UTR then an 870-nt CDS → `--cds 30:900`.

Step 3 — scan:

```
synatvis scan my_transgene.fasta --cds 30:900
```

Step 4 — read the report (top-down; flags ranked HIGH → INFO). Each flag has:

```
[HIGH ] cds    45-51   BsaI site (GGTCTC, + strand) in cds. Domesticate before MoClo assembly.
         evidence: Crozet 2018 (algal MoClo); Schroda 2019 ...
         fix     : synonymous: codon 5 GGT->GGC (G) [CTGGAGGGTCTCGCCAAG -> CTGGAGGGCCTCGCCAAG]
```

Step 5 — apply the fixes in your design tool, re-scan, repeat until the

HIGH/MEDIUM flags you care about are gone.

What each module checks

Module	Catches
polya	premature TGTAA poly(A) signal → truncated transcript (Cr uses TGTAA, not AAUAAA)
composition	low-GC troughs (silencing risk — low GC is the hazard in Cr) and homopolymers
codon	rare-codon clusters / low codon adaptation (affects elongation and mRNA stability)
cloning	internal Type IIS sites (BsaI/BsmBI/BbsI/SapI) for Golden Gate / MoClo
uorf	upstream AUGs competing with the main start
structure	missing/buried start codon, over-long or structured 5'UTR
instability	clustered AU-rich elements
splice	intended-intron sanity check; accidental splice pairs
silencing	heuristic only — GC troughs + strain/intron guidance (not a verdict)

5. Useful options

```

synatvis scan design.fasta --json           # machine-readable output
synatvis scan design.fasta --fail-on-high   # exit code 1 if any HIGH remains (for scripts/CI)
synatvis scan design.fasta --only polya,cloning # run selected modules only
synatvis scan design.fasta --exclude silencing # skip modules
synatvis profiles                           # list host profiles

```

6. Data provenance & tuning (for method development)

This build ships **authentic data**, not placeholders:

- **Codon table** (data/cr_nuclear_codon.tsv): real Kazusa *C. reinhardtii* nuclear codon usage (taxid 3055; 846 CDS; 420,455 codons; GC 66.3%).
- **Leg-1 negatives** (data/native_cr_cds.fasta): 500 real Cr nuclear CDS from NCBI RefSeq, ranked by codon adaptation (proxy for highly-expressed).
- **Leg-3 cases** (data/cases.yaml): real rescue pairs with NCBI accessions — native AT-rich GFP vs its Cr-codon-optimized twin, a real Cr gene with a BsaI site, and a real plastid gene (gated out).

The noisy modules' thresholds were **set from Leg 1** on the 500 real CDS (false-positive rates: polya ~2%, codon ~3%, composition ~1%). To retune on your own data:

```

synatvis validate leg1 --fasta your_native_cr_cds.fasta # false-positive rate
synatvis validate leg3 --cases your_cases.yaml          # sensitivity/specificity

```

Upgrade paths: a larger CoCoPUTs/Phytozome v6.1 codon table, an RNA-seq-ranked "highly-expressed" set, and more single-variable rescue pairs (same format).

7. Limitations (read before trusting a result)

- **Transcript-level only.** Silent on proteolysis / protein stability — a clean report is **not** a promise of expression.
 - **No score.** By design it never outputs an expression probability or grade.
 - **Cr nuclear only.** The chloroplast profile is a stub; other algae need their own profile. A nuclear report is meaningless for a chloroplast construct.
 - **Silencing is heuristic** and unvalidated — it flags only sequence-visible risk and points to strain (UVM4/UVM11) + intron mitigation.
-

8. Troubleshooting

Symptom	Fix
---------	-----

Python was not found (Windows)	Use <code>py</code> instead of <code>python</code> , or reinstall Python with "Add to PATH".
No module named <code>synatvis</code>	Run from the project folder , or do the Option B install.
<code>Activate.ps1</code> cannot be loaded	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , or call <code>.venv\Scripts\synatvis.exe</code> directly.
<code>pip install ".[full]"</code> fails on biopython/ViennaRNA	Those wheels lag new Python releases; install <code>".[yaml]"</code> instead — the core still works, GenBank/folding fall back to labelled heuristics.
Odd characters (■) in piped output	Cosmetic console-encoding only; the CLI writes UTF-8. Redirect to a file to confirm.

9. One-line summary to include with a submission

SynAT.Vis flags transcript-level failure modes (premature TGTAA poly(A) signals, GC-trough silencing risk, rare-codon clusters, Type IIS cloning sites, uORFs, AU-rich elements) in *Chlamydomonas reinhardtii* nuclear cassettes and proposes synonymous fixes. Diagnostic, not predictive; run `python selftest.py` to verify functionality on the included test kit.